

DSA Patterns Playbook

The DSA Pattern Playbook — How to Think, Not Just What to Memorize

0. The Meta-Framework — How to Approach ANY Problem

1. Two Pointers

2. Sliding Window

3. Fast & Slow Pointers (Floyd's Cycle Detection)

4. Merge Intervals

5. Cyclic Sort

6. In-place Reversal of Linked List

7. Tree BFS (Level Order)

8. Tree DFS (Preorder / Inorder / Postorder)

9. Two Heaps

10. Subsets / Backtracking

11. Modified Binary Search

12. Top K Elements (Heap)

13. K-way Merge

14. Topological Sort

15. Union-Find (Disjoint Set Union)

16. Trie (Prefix Tree)

17. Monotonic Stack / Queue

18. Prefix Sum (+ Difference Array)

19. Dynamic Programming — the biggest family, broken into sub-patterns

20. Greedy

21. Graph Algorithms (Weighted)

22. Matrix / Grid Traversal (Islands Pattern)

23. Bit Manipulation

Putting It Together — A Decision Flowchart

How to Actually Practice This (Mentor's Advice)

The DSA Pattern Playbook — How to Think, Not Just What to Memorize

The goal of this guide is not to give you 500 solutions. It's to build **pattern recognition** — so that when you see a new problem, your brain says “oh, this smells like sliding window” before you've even finished reading it.

0. The Meta-Framework — How to Approach ANY Problem

Before jumping to patterns, run this checklist every time:

1. **What's the input structure?** Array/string, linked list, tree, graph, matrix?
2. **Is it sorted or can I sort it?** Sorting unlocks two pointers, binary search.
3. **What's being asked?** Existence, count, min/max, all combinations, optimal value?
4. **What are the constraints (n)?** This tells you the expected time complexity:
 - $n \leq 10 \rightarrow$ exponential (2^n) fine $\rightarrow$ backtracking/brute force
 - $n \leq 20 \rightarrow$ bitmask DP fine
 - $n \leq 1,000 \rightarrow O(n^2)$ fine
 - $n \leq 10^5-10^6 \rightarrow O(n \log n)$ or $O(n)$ needed
 - $n \leq 10^9 \rightarrow O(\log n)$ or $O(1)$ needed (math/binary search)
5. **Do I need every element, or can I discard some greedily?**
6. **Is there overlapping subproblems / optimal substructure?** $\rightarrow$ DP.
7. **Am I looking for a contiguous chunk?** $\rightarrow$ sliding window / prefix sum.
8. **Am I looking for pairs/triplets with a condition?** $\rightarrow$ two pointers / hashing.

Once you answer these, 80% of the time the pattern picks itself.

1. Two Pointers

Recognize when: Sorted array/string, looking for pairs/triplets that satisfy a sum/condition, or comparing from both ends.

Idea: Use two indices moving toward each other (or same direction) instead of nested loops, exploiting sorted order to eliminate $O(n^2) \rightarrow O(n)$.

Why it works: In a sorted array, moving left pointer right increases the sum; moving right pointer left decreases it. You never need to re-check discarded combinations.

Template:

```
left, right = 0, n-1
while left < right:
    if condition_met: return/record
    elif sum_too_small: left += 1
    else: right -= 1
```

Classic problems: Two Sum (sorted), 3Sum, Container With Most Water, Trapping Rain Water, Valid Palindrome, Remove Duplicates.

Complexity: $O(n)$ time, $O(1)$ space — huge upgrade from $O(n^2)$ brute force.

2. Sliding Window

Recognize when: “Contiguous subarray/substring” + longest/shortest/count with a condition (sum, distinct chars, at most K, etc.)

Idea: Maintain a window [left, right]. Expand right to include elements; shrink left when the window breaks the constraint. Every element enters and leaves the window at most once $\rightarrow O(n)$.

Two flavors: - **Fixed size window:** window size is given (e.g., “max sum of subarray of size k”). - **Variable size window:** grow until invalid, then shrink until valid again.

Template (variable):

```
left = 0
for right in range(n):
    add arr[right] to window state
    while window is invalid:
        remove arr[left] from window state
        left += 1
    update answer using current window [left, right]
```

Classic problems: Longest Substring Without Repeating Characters, Minimum Window Substring, Longest Substring with K Distinct Chars, Max Sum Subarray of size K, Fruits into Baskets.

Complexity: $O(n)$ time, $O(k)$ space (k = window content, e.g. hashmap of chars).

3. Fast & Slow Pointers (Floyd’s Cycle Detection)

Recognize when: Linked list problems, especially cycle detection, finding the middle, or “happy number”-style sequences.

Idea: Two pointers move at different speeds (1x and 2x). If there’s a cycle, the fast pointer *will* lap the slow one. If not, fast reaches the end first.

Why it works: In a cycle of length C , the gap between fast and slow shrinks by 1 each step (relative speed), so they must meet within C steps.

Template:

```
slow = fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
if slow == fast: cycle detected
```

Classic problems: Linked List Cycle, Find Cycle Start, Middle of Linked List, Happy Number, Palindrome Linked List.

Complexity: $O(n)$ time, $O(1)$ space — beats $O(n)$ space hash-set approach.

4. Merge Intervals

Recognize when: Problems involving overlapping ranges/intervals (meetings, time slots, ranges).

Idea: Sort intervals by start time. Walk through; if current overlaps with the last merged one, merge (extend the end); otherwise start a new interval.

Template:

```
sort intervals by start
result = [intervals[0]]
for interval in intervals[1:]:
    if interval.start <= result[-1].end:
        result[-1].end = max(result[-1].end, interval.end)
    else:
        result.append(interval)
```

Classic problems: Merge Intervals, Insert Interval, Meeting Rooms I/II, Non-overlapping Intervals.

Complexity: $O(n \log n)$ due to sort, $O(n)$ space.

5. Cyclic Sort

Recognize when: Array contains numbers in range $[1, n]$ or $[0, n-1]$ and you need to find missing/duplicate numbers in **$O(n)$ time, $O(1)$ space**.

Idea: Since values map directly to indices, place each number at its “correct” index by swapping. After one pass, any index where `arr[i] != i+1` reveals missing/duplicate numbers.

Template:

```
i = 0
while i < n:
    correct = arr[i] - 1
    if arr[i] != arr[correct]:
        swap(arr[i], arr[correct])
    else:
        i += 1
```

Classic problems: Find Missing Number, Find All Duplicates, Find the Duplicate Number, First Missing Positive.

Complexity: $O(n)$ time, $O(1)$ space — this is the “aha” trick interviewers love because most people default to a hashset ($O(n)$ space).

6. In-place Reversal of Linked List

Recognize when: Need to reverse a linked list (fully or in groups/sub-ranges) without extra space.

Idea: Iteratively re-point `next` pointers using a `prev` tracker, moving one node at a time.

Template:

```
prev = None
curr = head
while curr:
    next_node = curr.next
    curr.next = prev
    prev = curr
    curr = next_node
return prev # new head
```

Classic problems: Reverse Linked List, Reverse in K-Groups, Reverse Sublist (between positions m and n), Swap Nodes in Pairs.

Complexity: O(n) time, O(1) space.

7. Tree BFS (Level Order)

Recognize when: “Level by level” traversal, shortest path in unweighted tree/graph, or need level-wise aggregation (min/max/sum per level).

Idea: Use a queue. Process one full level at a time by tracking the queue’s size at the start of the level.

Template:

```
queue = [root]
while queue:
    level_size = len(queue)
    for _ in range(level_size):
        node = queue.pop(0)
        process(node)
        if node.left: queue.append(node.left)
        if node.right: queue.append(node.right)
```

Classic problems: Level Order Traversal, Zigzag Traversal, Min Depth of Tree, Connect Level Order Siblings.

Complexity: O(n) time, O(w) space where w = max width of tree.

8. Tree DFS (Preorder / Inorder / Postorder)

Recognize when: Path-related problems (root-to-leaf sum, path exists), needing to explore “depth-first” before backtracking, or subtree computations.

Idea: Recursively go deep before wide. Choose traversal order based on when you need to “visit” the node relative to its children: -

Preorder (node → left → right): copying/serializing trees. - **Inorder** (left → node → right): BSTs → gives sorted order. - **Postorder** (left → right → node): when children’s results are needed first (e.g., height, deletion).

Template:

```
def dfs(node, path_sum):
```

```

if not node: return
path_sum += node.val
if is_leaf(node) and path_sum == target: record()
dfs(node.left, path_sum)
dfs(node.right, path_sum)

```

Classic problems: Path Sum I/II, Diameter of Binary Tree, Lowest Common Ancestor, Validate BST, Max Path Sum.

Complexity: $O(n)$ time, $O(h)$ space (h = height, due to recursion stack).

9. Two Heaps

Recognize when: Need to find median of a stream, or need to divide data into two halves and track a boundary value dynamically.

Idea: Maintain a **max-heap** for the smaller half and a **min-heap** for the larger half. Keep them balanced in size. The median is always at the top of one/both heaps.

Template:

```

max_heap (smaller half), min_heap (larger half)
on insert(num):
    push to appropriate heap based on comparison with tops
    rebalance so sizes differ by at most 1
median = top of larger heap, or avg of both tops if equal size

```

Classic problems: Find Median from Data Stream, Sliding Window Median, IPO (maximize capital), Scheduling problems.

Complexity: $O(\log n)$ per insertion, $O(1)$ to get median.

10. Subsets / Backtracking

Recognize when: “Find all combinations/permutations/subsets,” or a problem asks for **all** valid configurations (not just one optimal value), especially with constraints (N-Queens, Sudoku).

Idea: Build a solution incrementally; at each step, try every choice, recurse, and **undo (backtrack)** the choice before trying the next one. This explores the decision tree fully while reusing state.

Template:

```

def backtrack(path, choices):
    if is_complete(path): record(path); return
    for choice in choices:
        if is_valid(choice):
            path.append(choice)
            backtrack(path, remaining_choices)
            path.pop() # undo - this is the key step

```

Classic problems: Subsets, Permutations, Combination Sum, N-Queens, Word Search, Sudoku Solver, Generate Parentheses.

Complexity: Exponential in general ($O(2^n)$ or $O(n!)$), but pruning (early `is_valid` checks) makes it practical for $n \leq \sim 20$.

11. Modified Binary Search

Recognize when: Sorted (or rotated-sorted, or “sorted in an unusual way”) array, and you need $O(\log n)$. Also: “find boundary/first/last occurrence,” or searching an answer space (binary search on the answer).

Idea: Standard binary search finds an exact value. The “modified” versions search for a **boundary condition** — the first/last index satisfying a predicate, or search across a monotonic answer space rather than an array.

Template (search on answer space):

```
lo, hi = min_possible, max_possible
while lo < hi:
    mid = (lo + hi) // 2
    if feasible(mid): hi = mid      # or lo = mid+1, depending on direction
    else: lo = mid + 1
return lo
```

Classic problems: Search in Rotated Sorted Array, Find First/Last Position, Median of Two Sorted Arrays, Koko Eating Bananas, Capacity to Ship Packages, Search in 2D Matrix.

Complexity: $O(\log n)$ time — the biggest tell is “can I binary search on the ANSWER instead of the array?”

12. Top K Elements (Heap)

Recognize when: “Find the K largest/smallest/most frequent” elements.

Idea: Maintain a heap of size K. For “K largest,” use a **min-heap** of size K (so the smallest of your top-K is at the top, ready to be evicted). This avoids sorting the entire array ($O(n \log n)$) in favor of $O(n \log k)$.

Template:

```
min_heap = []
for num in array:
    push(num, min_heap)
    if len(min_heap) > k: pop smallest
return min_heap # contains K largest elements
```

Classic problems: Kth Largest Element, Top K Frequent Elements, K Closest Points to Origin, Reorganize String.

Complexity: $O(n \log k)$ time, $O(k)$ space — better than full sort when $k \ll n$.

13. K-way Merge

Recognize when: Multiple sorted lists/arrays need to be merged, or you need the smallest/kth element across several sorted sequences.

Idea: Use a min-heap holding one element (with its source list index) from each list. Pop min, push the next element from that same list. Repeat.

Template:

```
heap = [(list[i][0], i, 0) for i in range(k)] # heapify
while heap:
    val, list_idx, elem_idx = pop min
    add val to result
    if elem_idx+1 < len(lists[list_idx]):
        push (lists[list_idx][elem_idx+1], list_idx, elem_idx+1)
```

Classic problems: Merge K Sorted Lists, Kth Smallest in M Sorted Lists, Smallest Range Covering K Lists.

Complexity: $O(n \log k)$ where n = total elements, k = number of lists.

14. Topological Sort

Recognize when: Directed graph with **dependencies** (“A must happen before B”) — task scheduling, course prerequisites, build systems.

Idea: Order nodes so every directed edge $u \rightarrow v$ has u before v . Two approaches: - **Kahn’s algorithm (BFS):** repeatedly remove nodes with in-degree 0. - **DFS-based:** postorder DFS, then reverse.

Template (Kahn’s):

```
compute in-degree of every node
queue = nodes with in-degree 0
while queue:
    node = queue.pop()
    result.append(node)
    for neighbor in graph[node]:
        in_degree[neighbor] -= 1
        if in_degree[neighbor] == 0: queue.append(neighbor)
if len(result) != total_nodes: cycle exists → no valid ordering
```

Classic problems: Course Schedule I/II, Alien Dictionary, Build Order, Task Scheduling.

Complexity: $O(V + E)$ time. Bonus: this pattern also **detects cycles** — if you can’t process all nodes, a cycle exists.

15. Union-Find (Disjoint Set Union)

Recognize when: Need to dynamically group elements into connected components, detect cycles in an **undirected** graph, or answer “are these two connected?” queries efficiently.

Idea: Maintain a forest where each set has a representative “root.” `union(a,b)` merges two sets; `find(a)` returns the root. Use **path compression** + **union by rank** to get near- $O(1)$ operations.

Template:

```
parent = [i for i in range(n)]
def find(x):
```



```

if parent[x] != x:
    parent[x] = find(parent[x]) # path compression
return parent[x]
def union(a, b):
    ra, rb = find(a), find(b)
    if ra != rb: parent[ra] = rb

```

Classic problems: Number of Connected Components, Redundant Connection, Accounts Merge, Number of Islands II, Kruskal's MST.

Complexity: $O(\alpha(n))$ per operation (effectively constant — α is the inverse Ackermann function).

16. Trie (Prefix Tree)

Recognize when: Repeated **prefix** lookups on a set of strings — autocomplete, spell-check, word search in a grid, longest common prefix.

Idea: Tree where each path from root spells out a prefix; each node has children per character. Lets you check “does any word start with X?” in $O(\text{length of } X)$ instead of scanning all words.

Template:

```

class TrieNode:
    children = {}
    is_end = False

def insert(word):
    node = root
    for ch in word:
        node = node.children.setdefault(ch, TrieNode())
    node.is_end = True

```

Classic problems: Implement Trie, Word Search II, Autocomplete System, Longest Word in Dictionary, Replace Words.

Complexity: $O(L)$ per insert/search (L = word length), $O(\text{total characters})$ space.

17. Monotonic Stack / Queue

Recognize when: “Next greater/smaller element,” or need to track a max/min over a sliding window efficiently.

Idea: Maintain a stack (or deque) that’s always increasing or decreasing. When a new element violates the order, pop elements — each pop resolves that popped element’s “next greater/smaller” answer permanently.

Template (next greater element):

```

stack = [] # holds indices, values decreasing
for i, num in enumerate(arr):
    while stack and arr[stack[-1]] < num:
        idx = stack.pop()
        result[idx] = num # num is "next greater" for idx
    stack.append(i)

```

Classic problems: Next Greater Element, Daily Temperatures, Largest Rectangle in Histogram, Sliding Window Maximum, Trapping Rain Water (stack version).

Complexity: $O(n)$ — each element pushed and popped at most once, even though it looks like nested loops.

18. Prefix Sum (+ Difference Array)

Recognize when: Repeated **range sum queries**, or need cumulative totals, or subarray-sum-equals-K type problems.

Idea: Precompute `prefix[i] = sum(arr[0..i])`. Then `sum(l, r) = prefix[r] - prefix[l-1]` in $O(1)$. Combine with a hashmap for “count subarrays with sum K.”

Template (subarray sum = k):

```
prefix_sum = 0
count_map = {0: 1}
result = 0
for num in arr:
    prefix_sum += num
    result += count_map.get(prefix_sum - k, 0)
    count_map[prefix_sum] = count_map.get(prefix_sum, 0) + 1
```

Classic problems: Range Sum Query, Subarray Sum Equals K, Product of Array Except Self, Continuous Subarray Sum.

Complexity: $O(n)$ preprocessing, $O(1)$ per query.

19. Dynamic Programming — the biggest family, broken into sub-patterns

Recognize when: “Count the number of ways,” “find min/max cost,” AND the problem has **overlapping subproblems** (same subproblem solved repeatedly) + **optimal substructure** (optimal solution built from optimal solutions to subproblems).

How to think about DP (always in this order): 1. Define the state: what does `dp[i]` (or `dp[i][j]`) *mean* in plain English? 2. Find the recurrence: how does `dp[i]` relate to smaller states? 3. Base case(s). 4. Decide: top-down (memoized recursion) or bottom-up (iterative table)? 5. Can you compress space (rolling array) once it works?

Sub-patterns to recognize:

- **0/1 Knapsack** (pick or skip each item once): “subset sum,” “partition equal subset,” “target sum.” `dp[i][w] = max(dp[i-1][w], dp[i-1][w-wt[i]] + val[i])`
- **Unbounded Knapsack** (items reusable): “coin change,” “rod cutting.” `dp[w] = max(dp[w], dp[w-wt[i]] + val[i])` — inner loop stays on same row.

- **Longest Common Subsequence (LCS) family:** “edit distance,” “longest palindromic subsequence,” diff tools. $dp[i][j] = dp[i-1][j-1] + 1$ if match, else $\max(dp[i-1][j], dp[i][j-1])$.
- **Longest Increasing Subsequence (LIS) family:** $O(n^2)$ DP or $O(n \log n)$ with binary search + patience sorting.
- **Palindromic subsequence/substring:** DP over $dp[i][j]$ = is substring $i..j$ a palindrome, expand from center as alternative.
- **Matrix Chain / Interval DP:** “minimum cost to merge/cut,” burst balloons, $dp[i][j]$ over a range, iterate over a split point k .
- **State machine DP:** “best time to buy/sell stock” variants — track states like holding vs not holding.
- **DP on Trees:** postorder traversal returning (with_node, without_node) — house robber III.
- **Bitmask DP:** $n \leq \sim 20$, “visit all states/cities” — TSP, assign tasks. State = bitmask of visited items.

Complexity: Usually $O(\text{states} \times \text{transitions})$. The whole game is minimizing state space and transition cost.

20. Greedy

Recognize when: Making the **locally optimal choice at each step** provably leads to a globally optimal solution. Usually involves sorting first, then a single pass.

How to know if greedy applies (vs DP): Try to find a counterexample where the greedy/local choice fails. If you can’t, and you can informally *prove* the exchange argument (“if the optimal solution didn’t do this, we could swap to make it at least as good”), greedy is safe.

Classic problems: Activity Selection, Jump Game, Gas Station, Job Sequencing, Huffman Encoding, Minimum Platforms, Fractional Knapsack.

Complexity: Usually $O(n \log n)$ (dominated by the sort), $O(n)$ space.

21. Graph Algorithms (Weighted)

Recognize when: Weighted graph, need shortest path or minimum spanning tree.

- **Dijkstra’s (shortest path, non-negative weights):** min-heap + relax edges. $O((V+E) \log V)$.
- **Bellman-Ford (shortest path, handles negative weights, detects negative cycles):** relax all edges $V-1$ times. $O(V \cdot E)$.
- **Floyd-Warshall (all-pairs shortest path):** $dp[i][j] = \min(dp[i][j], dp[i][k] + dp[k][j])$ for all k . $O(V^3)$.
- **Prim’s / Kruskal’s (Minimum Spanning Tree):** Kruskal = sort

edges + Union-Find; Prim = min-heap growing from a start node.

Classic problems: Network Delay Time, Cheapest Flights Within K Stops, Min Cost to Connect All Points.

22. Matrix / Grid Traversal (Islands Pattern)

Recognize when: 2D grid, need to explore connected regions (4-directional or 8-directional).

Idea: BFS or DFS from each unvisited “land” cell, marking visited cells, counting/measuring each connected region.

Template:

```
def dfs(r, c):
    if out_of_bounds(r,c) or visited[r][c] or grid[r][c]==0: return
    visited[r][c] = True
    for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
        dfs(r+dr, c+dc)

for each cell:
    if not visited and grid[r][c]==1:
        dfs(r,c); count += 1
```

Classic problems: Number of Islands, Max Area of Island, Surrounded Regions, Rotting Oranges (BFS), Flood Fill.

Complexity: $O(\text{rows} \times \text{cols})$.

23. Bit Manipulation

Recognize when: Problem mentions “without extra space,” XOR tricks, subsets via bitmask, or powers of two.

Key tricks to memorize: - $x \& (x-1)$ clears the lowest set bit — useful for counting set bits. - $x \& (-x)$ isolates the lowest set bit. - XOR of a number with itself is 0 → find the single non-duplicate number. - $1 \ll i$ to represent/check subset membership in bitmask DP.

Classic problems: Single Number, Counting Bits, Sum of Two Integers (no + operator), Subsets via bitmask.

Putting It Together — A Decision Flowchart

```
Is it a linked list?
├─ cycle/middle → Fast & Slow Pointers
└─ reverse → In-place Reversal

Is it a tree?
├─ level-by-level → BFS
└─ path/subtree logic → DFS

Is it a graph?
├─ dependency order → Topological Sort
├─ shortest path (weighted) → Dijkstra/Bellman-Ford
├─ connectivity → Union-Find
└─ grid → Islands (BFS/DFS)

Is it an array/string?
├─ sorted + pair/triplet sum → Two Pointers
```

- └ contiguous subarray/substring → Sliding Window
- └ range sum queries → Prefix Sum
- └ next greater/smaller → Monotonic Stack
- └ numbers in range [1,n] → Cyclic Sort
- └ sorted, need $O(\log n)$ → Modified Binary Search
- └ overlapping intervals → Merge Intervals
- └ all combinations/permutations → Backtracking

Need top/kth element repeatedly? → Heap (Top K / Two Heaps)
Need to merge sorted sequences? → K-way Merge
Need prefix lookups on strings? → Trie
"Count ways" / "min-max cost" + overlapping subproblems? → DP
Local choice provably optimal? → Greedy

How to Actually Practice This (Mentor's Advice)

1. **Don't jump to code.** First say out loud: "This is a ___ pattern because ___." If you can't finish that sentence, you don't understand the problem yet.
2. **Solve the brute force first**, even mentally. It reveals *why* the optimal pattern works (usually by removing redundant re-computation).
3. **After solving, ask: "What made this pattern applicable?"**
Write that trigger phrase down. Over time you'll build your own personal cheat-sheet of *signals* → *pattern*, which is more valuable than mine.
4. **Redo problems after a week.** Recognition is muscle memory, not one-time understanding.
5. Group your practice **by pattern**, not randomly — do 5 sliding window problems back to back before moving on. Mixing too early slows recognition-building.

Good luck — and if you want, tell me a specific problem and I'll walk through *how* to recognize its pattern step-by-step, mentor-style, rather than just handing you the answer.